requirement, you can reuse the requirement for other policies with different minimum-age requirements.

Listing 24.6 The parameterized `MinimumAgeRequirement`

```
public class MinimumAgeRequirement : IAuthorizationRequirement      ❶
{
    public MinimumAgeRequirement(int minimumAge)                    ❷
    {
        MinimumAge = minimumAge;
    }
    public int MinimumAge { get; }                                  ❸
}
```

❶ The interface identifies the class as an authorization requirement.

❷ The minimum age is provided when the requirement is created.

❸ Handlers can use the exposed minimum age to determine whether the requirement is satisfied.

The requirements are the easy part. They represent each of the components of the policy that must be satisfied for the policy to be satisfied overall. Note that requirements are meant to be lightweight objects that can be created "manually." So while you can have constructor parameters, as shown in listing 24.6, you can't use dependency injection (DI) here. That's not as limiting as it sounds, because your handlers *can* use DI.

Creating a policy with multiple requirements

You've created the two requirements, so now you can configure the `"CanAccessLounge"` policy to use them. You configure your policies as you did before, in Program.cs. Listing 24.7 shows how to do this by creating an instance of each requirement and passing them to `AuthorizationPolicyBuilder`. The authorization handlers use these requirement objects when attempting to authorize the policy.